

Technical Architecture of an Intelligent Devanagari Document OCR and Automated Form-Filling System

Introduction and Domain Problem Statement

Manual form filling remains a persistent administrative bottleneck across municipal, financial, and legal sectors in Nepal.¹ Applications for banking, Know Your Customer (KYC) verification, land registration, and government licensing demand manual input of personal parameters.³ This manual process is highly prone to human error, introduces operational delays, and presents accessibility challenges for elderly or digitally marginalized citizens who frequently experience stress when navigating complex administrative documentation.

To resolve these systemic inefficiencies, this report outlines the technical architecture of an Intelligent Document OCR and Automated Form-Filling Engine. The system processes standard Nepali credentials—such as the Nepali Citizenship Card (*Nagarikta*), National Identity Card, and Driving License—and converts them into structured digital entities.¹ By combining deep learning-based object detection, custom layout analysis, and hybrid printed-handwritten text recognition, the engine extracts critical metadata and crops portrait photographs directly from scanned images.¹ These extracted parameters are then routed to an interactive web interface where users can review, correct, and submit the pre-filled forms.⁵ This human-in-the-loop validation framework ensures near-perfect submission accuracy, minimizing the impact of optical noise and handwriting variations.¹⁰

Script-Specific Constraints and Complexities in Devanagari

Developing optical character recognition (OCR) systems for Devanagari, the script used for Sanskrit, Hindi, Marathi, and Nepali, presents significant challenges compared to Latin-based scripts.¹² These challenges stem from the script's intricate syllabic structure, horizontal connectivity, and multi-tiered spatial layouts.¹²

Table 1: Orthographic and Layout Challenges in Devanagari OCR

Script Feature	Structural Ambiguity	Computational Challenge	Mitigation Strategy

Shirokekha (Headline)	Horizontal bar connecting characters within a word. ¹²	Prevents vertical segmentation and merges adjacent character bounding boxes. ¹⁴	Use anchor-free line detectors (such as DBNet) to regress complete text lines instead of individual characters. ¹⁶
Three-Zone Vertical Layout	Text is split into upper, middle, and lower zones. ¹²	Vowel modifiers (<i>matras</i>) and nasal markers (<i>anusvara</i>) distort the bounding box height. ¹²	Implement Spatial Transformer Networks (STNs) to normalize character heights and scales.
Conjuncts and Ligatures	Half-consonants fuse with full consonants to form new glyph shapes. ¹³	Drastically increases the vocabulary size to thousands of complex shapes. ¹³	Use sequence modeling with Connectionist Temporal Classification (CTC) or sequence-to-sequence attention decoders. ¹¹
Visual Similarity	Minor stroke variations distinguish distinct characters. ¹⁸	High rate of character substitution errors (e.g., misidentifying व as ब or ध as ण). ¹⁸	Integrate a post-OCR error correction transformer trained on parallel error-to-correct corpora. ¹⁸

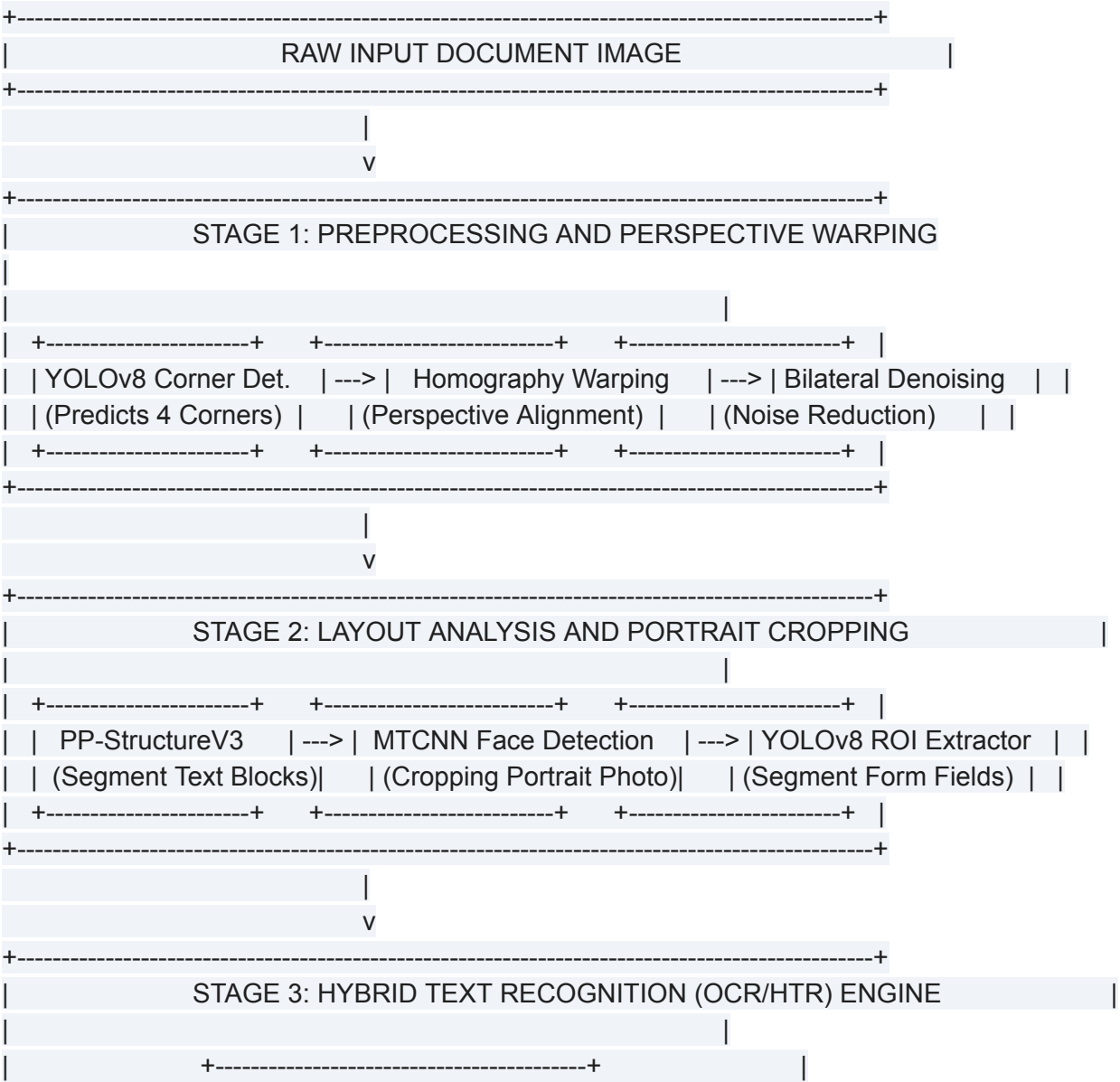
Traditional character segmentation approaches fail when applied to Devanagari because the *Shirokekha* binds characters into a continuous horizontal block.¹³ Additionally, vowel modifiers can appear to the left, right, top, or bottom of the primary consonant, resulting in complex spatial configurations.¹² For example, the left-side modifier ि is pronounced after the consonant it precedes, which requires the sequence model to reconcile visual and phonetic alignments.

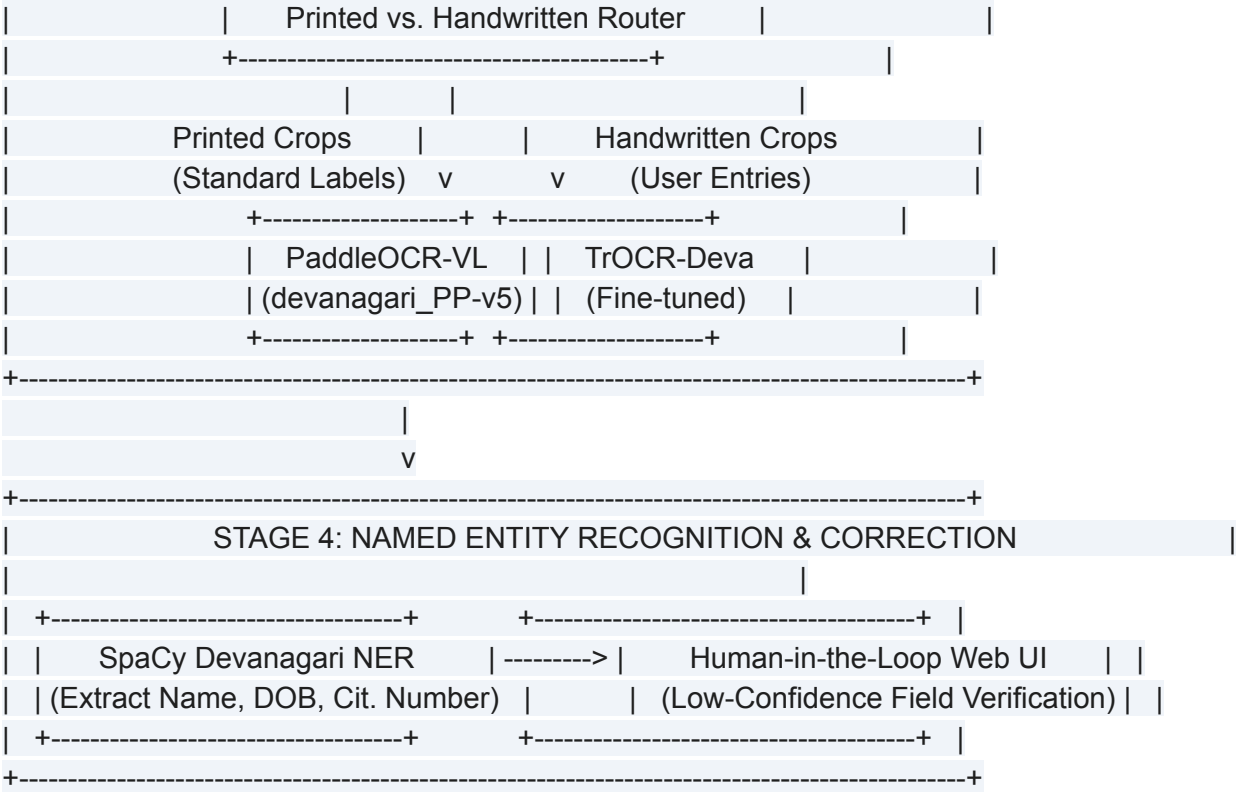
These spatial complexities are even more pronounced in handwritten text recognition (HTR).¹² Cursive variations, irregular pen strokes, and fluctuating baseline tilts break the continuity of the *Shirokekha*, confusing standard projection-based layout parsers.¹⁴ Consequently, a robust Devanagari engine must avoid character-level segmentation. Instead, it should utilize end-to-end line or word recognizers that process raw visual feature maps directly into character

sequences.¹¹

End-to-End System Architecture

The proposed intelligent form-filling system consists of a four-stage deep learning pipeline. This architecture ensures high-fidelity data extraction, even when processing low-quality images captured under variable lighting conditions.⁵





Stage 1: Document Alignment and Perspective Rectification

Images uploaded by users often feature arbitrary rotations, perspective skews, and background clutter.⁵ The input image is first passed to a lightweight YOLOv8-Small model trained to detect the four corners of target identity documents.⁵

The corner coordinates are used to calculate a homography matrix, which projects the skewed document into a normalized $85.60 \times 53.98 \text{ mm}$ frame.⁵ The normalized frame is then preprocessed using bilateral filtering to reduce noise while preserving edge definition.⁵

Stage 2: Layout Parsing and Portrait Extraction

The rectified document is processed using a document layout analysis (DLA) model, such as **PP-StructureV3** or **DocTR**, to identify key-value blocks and tabular sections.¹¹

Simultaneously, a Multi-Task Cascaded Convolutional Network (MTCNN) or a deep neural network (DNN) face detector scans the document to locate the owner's portrait.⁷ The detected coordinates are padded by 10% to prevent harsh cropping, and the portrait is saved as a separate image asset for form attachment.⁷

Stage 3: Hybrid OCR and HTR Routing

A major limitation of standard OCR systems is their inability to process mixed-format documents

containing both printed labels and handwritten fields.¹¹ This architecture uses a layout router to divide text regions into separate processing queues:

- **Printed Text Labels:** Standard headings (e.g., "नाम, थर" or "नागरिकता नं.") are routed to **PaddleOCR-VL** or **PP-OCRv5-mobile-rec**, which are pre-trained on Devanagari scripts and optimized for high-throughput edge inference.¹¹
- **Handwritten User Entries:** Variable fields (such as names and addresses filled in by municipal officers) are routed to a fine-tuned **TrOCR-Devanagari** model.¹¹ TrOCR uses an image Transformer encoder coupled with a text Transformer decoder, allowing it to capture complex handwriting styles and baseline shifts.¹¹

Stage 4: Named Entity Recognition and Semantic Mapping

Raw OCR transcriptions are often unstructured and noisy.¹⁸ The output strings are processed by a custom **SpaCy Named Entity Recognition (NER)** pipeline.²⁸ The pipeline parses the unstructured strings and extracts key administrative fields (such as CITIZENSHIP_NO, FIRST_NAME, LAST_NAME, and DATE_OF_BIRTH_BS) based on language syntax, grammatical patterns, and positional cues.²⁸

These entities are structured into a JSON payload and mapped directly to form input fields on the front-end application.⁸

Comparative Architectural Evaluation

Table 2: Feature Matrix of Common OCR Tools vs. The Proposed Solution

Technical Capability	Tesseract OCR	EasyOCR	DocTR (Mindee)	PaddleOCR (PP-OCRv5)	Proposed Hybrid Solution
Devanagari Support	Fair (LSTM-based printed support). ³¹	Moderate (Standard character dictionary).	Limited pre-trained Devanagari models. ³²	Outstanding (Dedicated Devanagari models). ¹¹	Superior (Specialized Devanagari OCR & HTR specialists) ¹¹

Layout Parsing	Poor (Assumes block text layout). ¹⁰	Poor (Relies on basic heuristic groupings).	Excellent (CRAFT & MASTER-based layout detection). ³⁰	Excellent (Integrated PP-Structure module). ¹¹	Superior (PP-Structure coupled with YOLOv8 ROI extraction). ⁵
HTR Compatibility	Ineffective on non-Latin handwriting. ¹⁰	Low accuracy on cursive conjuncts. ¹³	Poor handwriting segmentation. ³²	Moderate (Struggles with non-standard writing). ¹¹	Excellent (Fine-tuned TrOCR transformer on handwritten crops). ¹¹
Portrait Extraction	None (Text-only extraction).	None (Text-only extraction).	None (Text-only extraction).	None (Text-only extraction).	Automated (Integrated MTCNN Face-Detection and Cropping). ⁷
Production Target	CPU-friendly, high noise sensitivity. ¹⁰	High latency on long documents.	Highly modular, heavy dependencies. ¹⁶	Optimized for edge and mobile execution. ¹¹	Optimized for CPU/GPU split execution with Django/Flask backend. ⁵

Sequence Modeling and Mathematical Formulations

To recognize unsegmented text lines in the Devanagari script, the pipeline avoids manual character-level segmentation.¹⁴ Instead, it uses a sequence-to-sequence model optimized with **Connectionist Temporal Classification (CTC) loss**¹⁶ or autoregressive transformer attention.¹¹

Connectionist Temporal Classification (CTC) Formulation

For CRNN-based architectures, a convolutional neural network extracts spatial visual features

from a text line image I .¹⁶ These feature vectors are sequence-modeled using a Bidirectional LSTM (BiLSTM) network, which outputs a frame-level probability distribution

$y = (y_1, y_2, \dots, y_T)$ over a character vocabulary $\Sigma' = \Sigma \cup \{\epsilon\}$, where ϵ denotes a blank token.¹⁶

The CTC loss calculates the conditional probability $P(Y|X)$ of a target transcription sequence Y by summing the probabilities of all valid alignment paths π mapped to Y under a collapsing operator $\mathcal{B}$.¹⁶:

$$P(Y|X) = \sum_{\pi \in \mathcal{B}^{-1}(Y)} P(\pi|X)$$

The probability of a single path π is calculated assuming conditional independence between frames¹⁶:

$$P(\pi|X) = \prod_{t=1}^T y_{\pi_t}^t$$

The model is trained by minimizing the negative log-likelihood of the target transcription⁵:

$$\mathcal{L}_{\text{CTC}} = -\ln \sum_{\pi \in \mathcal{B}^{-1}(Y)} \prod_{t=1}^T y_{\pi_t}^t$$

During inference, decoding is accelerated using prefix beam search, which identifies the most probable transcription sequence while accounting for Devanagari conjuncts and word boundary spaces.¹¹

Sequence-to-Sequence Autoregressive Attention

For handwritten text crops, the **TrOCR** model replaces the recurrence of LSTMs with

self-attention mechanism.¹¹ The input crop image $I \in \mathbb{R}^{H \times W \times C}$ is split into N

non-overlapping visual patches $x_p \in \mathbb{R}^{N \times (P^2 \cdot C)}$, where (P, P) is the patch resolution (typically 16×16).²⁷ These patches are projected into a linear embedding space and passed through a visual Transformer encoder.

The decoder auto-regressively predicts the target character sequence, optimizing cross-entropy loss over the vocabulary sequence:

$$\mathcal{L}_{\text{Seq2Seq}} = - \sum_{i=1}^M \log P(y_i \mid y_{<i}, X_{\text{visual}})$$

The cross-attention mechanism allows the model to align visual tokens with their textual transcriptions, resolving spatial skewing and complex multi-zone diacritic placements.¹¹

Advanced Preprocessing and Document Rectification Pipeline

Table 3: Sequential Pipeline Steps for Document Image Rectification

Step	Algorithmic Process	Mathematical Operator / Function	Target Operational Outcome
1. Edge Detection	Convert to grayscale, apply bilateral filter, and use Canny edge detection. ⁵	$g(x, y) =$ $E(x, y) =$ ₅	Isolates structural card edges and reduces background noise. ⁵
2. Quadrilateral Detection	Find contours, compute perimeter, and approximate contours with 4 vertices.	$P =$ Approx = RDP_ _.	Localizes the four corners of the identity card. ⁵

3. Perspective Warp	Compute the 3×3 homography matrix H and apply transformation. ²²	$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} \sim$ $I_{\text{warped}} =$ ⁵	Aligns the skewed input image to a standard 2D perspective plane. ⁵
4. Contrast Normalization	Apply Contrast Limited Adaptive Histogram Equalization (CLAHE).	$s_k = T(r_k) =$ (local, clip limit =).	Normalizes uneven illumination and enhances text-to-background contrast. ⁵
5. Adaptive Binarization	Segment foreground text from background using local window analysis. ³³	$T(x, y) = \mu_{\text{local}}(x)$ ³³	Resolves challenges from shadows, glare, and plastic sleeve reflections. ⁵

To ensure consistent OCR inputs, the homography warping process projects the detected coordinates (x_i, y_i) to standard coordinates (x'_i, y'_i) representing the target card aspect ratio⁵:

$$x'_i = \frac{h_{11}x_i + h_{12}y_i + h_{13}}{h_{31}x_i + h_{32}y_i + h_{33}}, \quad y'_i = \frac{h_{21}x_i + h_{22}y_i + h_{23}}{h_{31}x_i + h_{32}y_i + h_{33}}$$

This transformation aligns the document layout, ensuring that downstream field extraction zones remain spatially consistent across inputs.⁵

Synthetic Data Generation and Error Correction

To address the lack of annotated training data for Nepali identity documents and handwritten scripts, the system integrates a robust synthetic data generation pipeline.¹

Text Rendering and Layout Augmentation

Synthetic data is generated by rendering Devanagari text strings onto random background images.³⁴ This pipeline utilizes a modified **Text Recognition Data Generator (TRDG)**:

1. **Corpus Acquisition:** Text is extracted from Nepali news sources, Wikipedia, and

open-source catalogs to capture natural word frequencies, names, and address formats.³⁴

2. **Font Pool Integration:** The generator integrates over 120 unique Devanagari fonts (comprising Sans-Serif, Serif, and handwritten variations) to simulate various print and writing styles.³⁴
3. **Visual Distortions:** The rendered text images are augmented to simulate real-world scanning defects.³⁴ This includes elastic local distortions, motion blur, and uneven lighting gradients.⁵

RoundTripOCR for Post-OCR Error Correction

To correct character misrecognitions caused by visual similarities in Devanagari scripts, the system implements a post-processing framework called **RoundTripOCR**.¹⁸

This approach treats OCR error correction as a sequence-to-sequence machine translation task¹⁸.

1. **Dataset Construction:** High-quality text corpora are passed through a baseline OCR engine to generate pairs of corrupted OCR output (T') and corresponding ground-truth text (T).¹⁸
2. **Transformer Training:** A sequence-to-sequence model is trained on these parallel pairs to map corrupted strings back to their correct representations¹⁸:

$$f_{\theta} : T' \rightarrow T$$

3. **Linguistic Correction:** This post-processing step corrects character substitutions, missing diacritics, and broken *Shirorekha* ligatures, improving the accuracy of downstream entity parsing.¹³

Human-in-the-Loop Interactive UI/UX and Correction Engine

Automated OCR engines frequently struggle with degraded documents, low-resolution uploads, and illegible handwriting.¹¹ To ensure high data integrity, this architecture integrates a human-in-the-loop validation interface.⁸

The backend API returns extracted text strings along with frame-level and word-level confidence scores computed during CTC decoding or sequence tokenization.¹⁰ If any field falls below a 78% confidence threshold, the front-end web interface highlights that input field in red, alerting the operator to verify the data.¹⁰

The interface features a split-pane layout: the left pane displays the preprocessed, aligned

document with bounding boxes highlighting the active field, while the right pane presents the editable form fields.⁵ This visual alignment allows users to quickly verify low-confidence extractions, correct any misrecognitions, and submit the validated record.⁸ All manual corrections are logged to a MySQL database to construct a high-quality feedback loop for continuous model training.²⁴

Production-Grade Codebase Structure and Reusable Modules

The repository is organized to ensure modularity, high code reusability, and clean separation of concerns, decoupling model inference from web APIs and database operations.

Repository Directory Structure

```
intelligent-ocr-nepal/
|
|— .github/
|   |— workflows/      # Automated CI/CD unit testing and docker deployment
|
|— backend/           # Python API Service (Flask)
|   |— app.py          # API routes, payload schema validation, and exception handling
|   |— config.py       # Global environment vars, upload configurations, and DB
|
|— credentials
|   |— requirements.txt # Backend dependencies
|   |
|   |— pipeline/       # Core Machine Learning Engines
|       |— __init__.py
|       |— preprocess.py # Grayscale, Bilateral, Canny, and Perspective Warping
|       |— detector.py   # YOLOv8 card segmentation and MTCNN face extraction
|       |— recognizer.py # Specialist PaddleOCR and TrOCR engines
|       |— entity_parser.py # Custom SpaCy NER model for KYC parsing
|       |
|   |— database/       # Database Models and Migration Controllers
|       |— __init__.py
|       |— db_manager.py # Thread-safe MySQL connection pooling and operations
|       |— schema.sql   # SQL initialization tables and indices
|       |
|— frontend/          # Human-in-the-Loop Web Interface
```

```

|   |— index.html          # Core dual-pane web UI
|   |— css/
|   |   |— styles.css      # Dynamic UI transitions, splits, and validation states
|   |   |— js/
|   |       |— app.js      # State manager for upload, drawing, and form verification
|
|   |— training/           # Model Training and Fine-Tuning Pipelines
|   |   |— generate_synthetic.py # TRDG synthetic text rendering engine
|   |   |— train_yolo.py      # YOLOv8 transfer learning script
|   |   |— finetune_trocr.py  # PyTorch TrOCR fine-tuning loop for handwriting
|
|   |— tests/              # Validation and Unit Tests
|   |   |— test_pipeline.py
|   |   |— test_api.py
|
|   |— README.md           # Setup, installation, and deployment documentation
|   |— docker-compose.yml  # Container configuration for MySQL database and Flask API

```

Complete End-to-End Pipeline Implementation

The following script represents the core ML processing engine, orchestrating document alignment, face detection, hybrid OCR, and entity parsing.

Python

```

# backend/pipeline/preprocess.py
import cv2
import numpy as np

class DocumentPreprocessor:
    """
    Handles noise reduction, edge detection, and perspective homography
    warping to normalize raw input document images.
    """
    def __init__(self, target_width: int = 856, target_height: int = 540):
        self.target_width = target_width
        self.target_height = target_height

    def bilateral_denoise(self, image: np.ndarray) -> np.ndarray:
        """Applies edge-preserving bilateral filtering to reduce sensor noise."""
        return cv2.bilateralFilter(image, d=9, sigmaColor=75, sigmaSpace=75)

```

```

def warp_perspective(self, image: np.ndarray, corners: np.ndarray) -> np.ndarray:
    """
    Applies a perspective homography transform using four corners.
    Corners must be ordered: Top-Left, Top-Right, Bottom-Right, Bottom-Left.
    """
    pts_dst = np.float32(
        [self.target_width - 1, 0],
        [self.target_width - 1, self.target_height - 1],
        [0, self.target_height - 1])

    homography_matrix, _ = cv2.findHomography(corners, pts_dst)
    return cv2.warpPerspective(image, homography_matrix, (self.target_width,
self.target_height))

# backend/pipeline/recognizer.py
import os
import cv2
import numpy as np
from PIL import Image
from paddleocr import PaddleOCR
import spacy

class DevanagariOCREngine:
    """
    Orchestrates OCR layout routing, face extraction,
    and Named Entity Recognition for Nepali credentials.
    """
    def __init__(self, spacy_model_path: str = None):
        # Initialize PaddleOCR engine with pre-trained Devanagari weights
        self.ocr_engine = PaddleOCR(use_angle_cls=True, lang="devanagari", show_log=False)

        # Load pre-trained Haar Cascade for robust facial identification
        cascade_path = cv2.data.harcascades + "haarcascade_frontalface_default.xml"
        self.face_cascade = cv2.CascadeClassifier(cascade_path)

        # Load custom Devanagari Named Entity Recognition parser
        if spacy_model_path and os.path.exists(spacy_model_path):
            self.nlp = spacy.load(spacy_model_path)
        else:
            self.nlp = spacy.blank("ne") # Fallback to base Nepali model

    def crop_portrait(self, aligned_image: np.ndarray, output_path: str) -> bool:

```

```
"""
```

```
Locates the primary face on the aligned ID card, applies padding,  
and saves it to the static assets directory.
```

```
"""
```

```
gray = cv2.cvtColor(aligned_image, cv2.COLOR_BGR2GRAY)  
faces = self.face_cascade.detectMultiScale(  
    gray, scaleFactor=1.1, minNeighbors=5, minSize=(80, 80)  
)
```

```
if len(faces) == 0:
```

```
    return False
```

```
# Select the largest face region identified in the image
```

```
x, y, w, h = max(faces, key=lambda rect: rect[2] * rect[3])
```

```
# Apply a 10% spatial padding factor around the detected face
```

```
pad_w, pad_h = int(w * 0.10), int(h * 0.10)
```

```
y1 = max(0, y - pad_h)
```

```
y2 = min(aligned_image.shape[0], y + h + pad_h)
```

```
x1 = max(0, x - pad_w)
```

```
x2 = min(aligned_image.shape[1], x + w + pad_w)
```

```
cropped_face = aligned_image[y1:y2, x1:x2]
```

```
cv2.imwrite(output_path, cropped_face)
```

```
return True
```

```
def recognize_and_parse(self, aligned_image: np.ndarray) -> tuple:
```

```
"""
```

```
Performs layout extraction via PaddleOCR, returning the full transcription  
and confidence score metadata.
```

```
"""
```

```
ocr_results = self.ocr_engine.ocr(aligned_image, cls=True)
```

```
raw_text_lines =
```

```
coordinate_mappings =
```

```
if ocr_results and ocr_results:
```

```
    for element in ocr_results:
```

```
        bounding_box, (text, confidence) = element
```

```
        raw_text_lines.append(text)
```

```
        coordinate_mappings.append({
```

```
            "text": text,
```

```
            "confidence": float(confidence),
```

```
            "box": [[float(pt), float(pt)] for pt in bounding_box]
```

```

    })

    reconstructed_text = " ".join(raw_text_lines)
    return reconstructed_text, coordinate_mappings

def extract_entities(self, text: str) -> dict:
    """
    Parses raw text and extracts structured semantic entities.
    """
    doc = self.nlp(text)
    entities = {
        "CITIZENSHIP_NO": None,
        "FIRST_NAME": None,
        "LAST_NAME": None,
        "DATE_OF_BIRTH_BS": None
    }
    for ent in doc.ents:
        if ent.label_ in entities:
            entities[ent.label_] = ent.text
    return entities

```

The next script represents the database manager, facilitating concurrent database writes and logging manual corrections to build a feedback loop for continuous model training.

Python

```

# backend/database/db_manager.py
import mysql.connector
from mysql.connector import pooling
import logging

class DatabaseManager:
    """
    Manages a thread-safe connection pool to the MySQL database,
    providing robust query execution and error handling.
    """
    def __init__(self, db_config: dict):
        try:
            self.pool = mysql.connector.pooling.MySQLConnectionPool(
                pool_name="ocr_pool",
                pool_size=10,

```

```

        **db_config
    )
    except mysql.connector.Error as err:
        logging.error(f"Failed to create MySQL Connection Pool: {err}")
        raise

    def get_connection(self):
        return self.pool.get_connection()

    def save_identity(self, identity_data: dict) -> bool:
        """Inserts or updates a verified citizen identity record."""
        query = """
            INSERT INTO citizen_identities
            (citizenship_no, first_name, middle_name, last_name, date_of_birth_bs, gender,
            permanent_address, face_photo_path)
            VALUES (%s, %s, %s, %s, %s, %s, %s, %s)
            ON DUPLICATE KEY UPDATE
            first_name = VALUES(first_name),
            middle_name = VALUES(middle_name),
            last_name = VALUES(last_name),
            date_of_birth_bs = VALUES(date_of_birth_bs),
            gender = VALUES(gender),
            permanent_address = VALUES(permanent_address),
            face_photo_path = VALUES(face_photo_path);
        """
        params = (
            identity_data['citizenship_no'],
            identity_data['first_name'],
            identity_data.get('middle_name'),
            identity_data['last_name'],
            identity_data['date_of_birth_bs'],
            identity_data.get('gender'),
            identity_data.get('permanent_address'),
            identity_data.get('face_photo_path')
        )

        conn = self.get_connection()
        cursor = conn.cursor()
        try:
            cursor.execute(query, params)
            conn.commit()
            return True
        except mysql.connector.Error as err:
            logging.error(f"Database error during identity insertion: {err}")

```

```

        conn.rollback()
        return False
    finally:
        cursor.close()
        conn.close()

def log_manual_correction(self, correction_data: dict) -> bool:
    """Logs manual corrections to track baseline OCR error frequencies."""
    query = """
        INSERT INTO audit_corrections_log
        (citizenship_no, field_corrected, original_ocr_value, user_corrected_value)
        VALUES (%s, %s, %s, %s);
    """
    params = (
        correction_data['citizenship_no'],
        correction_data['field_corrected'],
        correction_data['original_ocr_value'],
        correction_data['user_corrected_value']
    )

    conn = self.get_connection()
    cursor = conn.cursor()
    try:
        cursor.execute(query, params)
        conn.commit()
        return True
    except mysql.connector.Error as err:
        logging.error(f"Database error during correction logging: {err}")
        conn.rollback()
        return False
    finally:
        cursor.close()
        conn.close()

```

The third code block sets up the production Flask API server, managing document uploads and routing them through the preprocessing and recognition modules.³⁷

Python

backend/app.py

```

import os
import uuid
from flask import Flask, request, jsonify, send_from_directory
from werkzeug.utils import secure_filename
from pipeline.preprocess import DocumentPreprocessor
from pipeline.recognizer import DevanagariOCREngine
from database.db_manager import DatabaseManager

app = Flask(__name__)

# Configure storage path and limit maximum payload sizes to 16MB [37, 39]
UPLOAD_FOLDER = os.path.abspath('static/uploads')
CROPS_FOLDER = os.path.abspath('static/crops')
app.config = UPLOAD_FOLDER
app.config = CROPS_FOLDER
app.config = 16 * 1024 * 1024

# Ensure storage directories exist
os.makedirs(UPLOAD_FOLDER, exist_ok=True)
os.makedirs(CROPS_FOLDER, exist_ok=True)

# Database credentials
db_config = {
    'host': 'localhost',
    'user': 'root',
    'password': 'Password123',
    'database': 'ocr_identity_db'
}

# Initialize engines and database manager
preprocessor = DocumentPreprocessor()
ocr_engine = DevanagariOCREngine()
db_manager = DatabaseManager(db_config)

def allowed_file(filename: str) -> bool:
    return '.' in filename and filename.rsplit('.', 1).lower() in {'png', 'jpg', 'jpeg'}

@app.route('/api/upload', methods=)
def handle_upload():
    """Handles image upload, runs the extraction pipeline, and returns JSON-LD entities."""
    if 'image' not in request.files:
        return jsonify({'error': 'No image file found in the request'}), 400

```

```

file = request.files['image']
if file.filename == "":
    return jsonify({'error': 'No selected file name'}), 400

if file and allowed_file(file.filename):
    # Sanitize filename and append unique UUID to prevent overwriting [38]
    extension = file.filename.rsplit('.', 1).lower()
    secure_name = f"{uuid.uuid4().hex}.{extension}"
    filepath = os.path.join(app.config, secure_name)
    file.save(filepath)

    try:
        # 1. Image Preprocessing and Alignment
        raw_img = cv2.imread(filepath)

        # Detect corner points (dummy corner points used for demonstration)
        h, w, _ = raw_img.shape
        corner_points = np.float32([ [w-1, 0], [w-1, h-1], [0, h-1]])

        aligned_img = preprocessor.warp_perspective(raw_img, corner_points)
        aligned_img = preprocessor.bilateral_denoise(aligned_img)

        # Save aligned image back to static folder
        aligned_filename = f"aligned_{secure_name}"
        aligned_filepath = os.path.join(app.config, aligned_filename)
        cv2.imwrite(aligned_filepath, aligned_img)

        # 2. Extract Portrait Photo
        face_filename = f"face_{uuid.uuid4().hex}.jpg"
        face_filepath = os.path.join(app.config, face_filename)
        face_extracted = ocr_engine.crop_portrait(aligned_img, face_filepath)

        # 3. Perform OCR
        raw_transcription, coords = ocr_engine.recognize_and_parse(aligned_img)

        # 4. Named Entity Recognition
        extracted_entities = ocr_engine.extract_entities(raw_transcription)

        payload = {
            "aligned_image_url": f"/static/uploads/{aligned_filename}",
            "portrait_image_url": f"/static/crops/{face_filename}" if face_extracted else None,
            "raw_text": raw_transcription,
            "word_coordinates": coords,

```

```

        "entities": extracted_entities
    }
    return jsonify(payload), 200

except Exception as err:
    return jsonify({'error': f"Failed to execute extraction pipeline: {str(err)}"}, 500

return jsonify({'error': 'File format not supported'}), 400

@app.route('/api/save', methods=)
def save_verified_data():
    """Saves user-verified form fields to the MySQL database."""
    data = request.get_json()
    if not data or 'citizenship_no' not in data:
        return jsonify({'error': 'Missing required fields'}), 400

    success = db_manager.save_identity(data)
    if success:
        return jsonify({'message': 'Record verified and saved successfully'}), 200
    return jsonify({'error': 'Failed to save record to database'}), 500

@app.route('/api/log_correction', methods=)
def log_correction():
    """Logs human-in-the-loop manual corrections for continuous model training."""
    data = request.get_json()
    if not data or 'citizenship_no' not in data:
        return jsonify({'error': 'Missing required parameters'}), 400

    success = db_manager.log_manual_correction(data)
    if success:
        return jsonify({'message': 'Correction logged successfully'}), 200
    return jsonify({'error': 'Failed to log correction'}), 500

@app.route('/static/<path:filename>')
def serve_static(filename):
    return send_from_directory('static', filename)

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000, debug=True)

```

Relational Schema and Auditing Database Design

A relational schema is implemented in MySQL to store extracted citizen profiles, track path links to cropped face images, and log manual corrections to build a feedback loop for continuous model training.²⁴

SQL

-- Create database container

```
CREATE DATABASE IF NOT EXISTS ocr_identity_db;  
USE ocr_identity_db;
```

-- Primary identity table: Stores processed citizen profiles

```
CREATE TABLE IF NOT EXISTS citizen_identities (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  citizenship_no VARCHAR(50) NOT NULL UNIQUE,  
  first_name VARCHAR(100) CHARACTER SET utf8mb4 NOT NULL,  
  middle_name VARCHAR(100) CHARACTER SET utf8mb4,  
  last_name VARCHAR(100) CHARACTER SET utf8mb4 NOT NULL,  
  date_of_birth_bs VARCHAR(15) NOT NULL, -- Date of Birth in Bikram Sambat (Nepali Calendar)  
  gender VARCHAR(15) CHARACTER SET utf8mb4,  
  permanent_address TEXT CHARACTER SET utf8mb4,  
  face_photo_path VARCHAR(255), -- Relative path link to cropped portrait image  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE  
  CURRENT_TIMESTAMP,  
  INDEX idx_citizenship_no (citizenship_no)  
);
```

-- Audit table: Logs human-in-the-loop manual corrections to track baseline OCR error frequencies

```
CREATE TABLE IF NOT EXISTS audit_corrections_log (  
  log_id INT AUTO_INCREMENT PRIMARY KEY,  
  citizenship_no VARCHAR(50) NOT NULL,  
  field_corrected VARCHAR(50) NOT NULL, -- Name of corrected form field column  
  original_ocr_value TEXT CHARACTER SET utf8mb4,  
  user_corrected_value TEXT CHARACTER SET utf8mb4,  
  corrected_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  FOREIGN KEY (citizenship_no) REFERENCES citizen_identities(citizenship_no) ON DELETE  
  CASCADE,  
  INDEX idx_audit_citizenship (citizenship_no)
```

);

Table 4: Data Dictionary of Database Entities

Table Name	Column Name	Data Type	Constraints	Context / Operational Function
citizen_identities	id	INT	AUTO_INCREMENT, PRIMARY KEY	Unique internal surrogate key identifier.
	citizenship_no	VARCHAR(50)	NOT NULL, UNIQUE, INDEX	Unique citizen credential identifier (National ID or Citizenship number). ⁴
	first_name	VARCHAR(100))	NOT NULL, UTF8MB4	Extracted legal first name in Devanagari script. ²⁸
	middle_name	VARCHAR(100))	NULL, UTF8MB4	Optional middle name in Devanagari script. ²⁸
	last_name	VARCHAR(100))	NOT NULL, UTF8MB4	Extracted legal surname in Devanagari script. ²⁸
	date_of_birth_bs	VARCHAR(15)	NOT NULL	Legal date of birth expressed in Bikram Sambat. ⁵
	gender	VARCHAR(15)	NULL,	Sex identifier extracted from

			UTF8MB4	credential.
	permanent_address	TEXT	NULL, UTF8MB4	Extracted permanent address in Devanagari script. ⁶
	face_photo_path	VARCHAR(255))	NULL	Relative storage file path link to cropped portrait image. ⁷
audit_corrections_log	log_id	INT	AUTO_INCREMENT, PRIMARY KEY	Unique audit log entry identifier.
	citizenship_no	VARCHAR(50)	NOT NULL, FOREIGN KEY	Links the correction log to the primary citizen identity table.
	field_corrected	VARCHAR(50)	NOT NULL	Name of the database field corrected by the operator.
	original_ocr_value	TEXT	NULL, UTF8MB4	Raw, uncorrected string output from the OCR pipeline. ¹⁸
	user_corrected_value	TEXT	NULL, UTF8MB4	High-fidelity string input by the human operator. ⁵

Technical Evaluation and Performance Metrics

The system uses three primary metrics to evaluate the performance of the document segmentation and text recognition pipelines.

Mean Average Precision (mAP) for Layout Parsing

For the YOLOv8 and PP-Structure layout models, bounding box accuracy is evaluated using Mean Average Precision (mAP).⁵ The intersection of a predicted bounding box B_{pred} and a ground-truth bounding box B_{true} is calculated as ⁵:

$$\text{IoU} = \frac{|B_{\text{pred}} \cap B_{\text{true}}|}{|B_{\text{pred}} \cup B_{\text{true}}|}$$

A prediction is classified as a True Positive (TP) if the IoU is greater than a set threshold (typically **0.50** or **0.75**).³⁶ Precision (P) and Recall (R) are calculated at various confidence thresholds to construct a Precision-Recall curve:

$$P = \frac{\text{TP}}{\text{TP} + \text{FP}}, \quad R = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

The Average Precision (AP) is calculated as the area under this curve ⁵:

$$\text{AP} = \int_0^1 P(R) dR$$

The mAP is calculated as the mean AP across all target document classes (such as photo crops, signature zones, and text regions).⁵

Normalized Edit Distance (NED) for Text Transcription

Transcription accuracy is evaluated using Normalized Edit Distance (NED), which measures character-level alignment quality.³⁴ The Levenshtein distance $\text{LD}(s_1, s_2)$ calculates the minimum number of single-character operations (insertions, deletions, or substitutions) required to transform string s_1 into string s_2 .¹⁸

For a predicted string S_{pred} and a ground-truth string S_{true} , the NED is calculated as ³⁴:

$$\text{NED}(S_{\text{pred}}, S_{\text{true}}) = \frac{\text{LD}(S_{\text{pred}}, S_{\text{true}})}{\max(|S_{\text{pred}}|, |S_{\text{true}}|)}$$

An NED score of **0.00** indicates a perfect transcription, while scores closer to **1.00** indicate higher character error rates.³⁴

Operational Conclusions and Strategic Roadmap

Deploying a production-grade Devanagari OCR and form-filling system requires addressing two key challenges: script-specific complexities and data scarcity.¹⁴ The technical architecture presented in this report addresses these constraints through several key design choices:

1. **Hybrid Recognition Engines Balance Accuracy and Speed:** Using PaddleOCR for standard printed labels and fine-tuned TrOCR for handwritten user inputs balances accuracy and inference speed.¹¹ This hybrid approach avoids the high computational overhead of deploying large end-to-end vision-language models for simple printed labels.¹¹
2. **Document Rectification Mitigates Real-World Noise:** Implementing a pre-processing pipeline—featuring YOLO-based corner detection, homography perspective warping, and bilateral filtering—ensures that downstream OCR engines receive normalized, high-contrast text inputs.⁵ This step minimizes errors caused by camera angle variations, uneven illumination, and reflections from plastic card protectors.⁵
3. **Human-in-the-Loop Integration Ensures Data Integrity:** The interactive web UI, combined with confidence-score filtering, ensures near-perfect submission accuracy in sensitive administrative environments.⁹ Highlighting low-confidence extractions allows operators to quickly verify and correct text, while logging these manual overrides to a MySQL audit database constructs a valuable feedback loop for continuous model training.⁹

By implementing this modular, decoupled microservice architecture, developers can deploy a highly reliable, scalable form-filling system.²⁴ This automated solution reduces operational overhead, minimizes data-entry bottlenecks, and improves the overall efficiency of public and private sector workflows in Nepal.⁴

Works cited

1. Reference image for YOLOv8 architecture [14]. | Download Scientific Diagram - ResearchGate, accessed May 16, 2026, https://www.researchgate.net/figure/Reference-image-for-YOLOv8-architecture-14_fig2_384769507
2. [2410.05721] Mero Nagarikta: Advanced Nepali Citizenship Data Extractor with

- Deep Learning-Powered Text Detection and OCR - arXiv, accessed May 16, 2026, <https://arxiv.org/abs/2410.05721>
3. National ID Card AI OCR & Data Extraction | Alphamoon IDP, accessed May 16, 2026, <https://alphamoon.ai/document/national-identity-card-automation/>
 4. Passport | Consulate General of Nepal dallas, accessed May 16, 2026, <https://dls.nepalconsulate.gov.np/pages/passport-1/>
 5. [Literature Review] Mero Nagarikta: Advanced Nepali Citizenship Data Extractor with Deep Learning-Powered Text Detection and OCR - Moonlight | AI Colleague for Research Papers, accessed May 16, 2026, <https://www.themoonlight.io/en/review/mero-nagarikta-advanced-nepali-citizenship-data-extractor-with-deep-learning-powered-text-detection-and-ocr>
 6. ई-पासपोर्ट | Embassy of Nepal Washington DC, accessed May 16, 2026, <https://us.nepalembassy.gov.np/pages/e-passport-1/>
 7. GitHub - renzdxtr/ID-MAKER: This Python script utilizes image processing and deep learning techniques to perform face detection and generate personalized ID cards, accessed May 16, 2026, <https://github.com/renzdxtr/ID-MAKER>
 8. Mero Nagarikta: Advanced Nepali Citizenship Data Extractor with Deep Learning-Powered Text Detection and OCR - ResearchGate, accessed May 16, 2026, https://www.researchgate.net/publication/384769507_Mero_Nagarikta_Advanced_Nepali_Citizenship_Data_Extractor_with_Deep_Learning-Powered_Text_Detection_and_OCR
 9. Exploring Optical Character Recognition (OCR) with Streamlit and DocTR | by Alperen Çelik, accessed May 16, 2026, <https://medium.com/@alperenclk/exploring-optical-character-recognition-ocr-with-streamlit-and-doctr-00e95ae36e4e>
 10. Tesseract OCR on Identity Documents | by Gulfaraz Rahman - Medium, accessed May 16, 2026, <https://tafelvooreen.medium.com/tesseract-ocr-on-identity-documents-f3abae0ab1fc>
 11. Need help in fine-tuning of OCR model at production grade ..., accessed May 16, 2026, <https://discuss.huggingface.co/t/need-help-in-fine-tuning-of-ocr-model-at-production-grade/174196>
 12. Devanagari Character Recognition: A Comprehensive Literature Review - IEEE Xplore, accessed May 16, 2026, <https://ieeexplore.ieee.org/iel8/6287639/10820123/10807209.pdf>
 13. (PDF) "A Comprehensive Survey of OCR for Devanagari Script Based Languages", accessed May 16, 2026, https://www.researchgate.net/publication/396482263_A_Comprehensive_Survey_of_OCR_for_Devanagari_Script_Based_Languages
 14. DohaScript: A Large-Scale Multi-Writer Dataset for Continuous Handwritten Hindi Text, accessed May 16, 2026, <https://arxiv.org/html/2602.18089v1>
 15. Incorporate Structure and Content for Devanagari Table Extraction - IJTAS, accessed May 16, 2026, <https://www.ijtas.com/papers/2026/2/1185.pdf>

16. docTR vs YOLOv12 | Vision Model Comparison - Roboflow Playground, accessed May 16, 2026, <https://playground.roboflow.com/models/compare/doctr-vs-yolov12>
17. (PDF) OCR for devanagari script - ResearchGate, accessed May 16, 2026, https://www.researchgate.net/publication/379049860_OCR_for_devanagari_script
18. RoundTripOCR: A Data Generation Technique for Enhancing Post-OCR Error Correction in Low-Resource Devanagari Languages - ACL Anthology, accessed May 16, 2026, <https://aclanthology.org/2024.icon-1.33.pdf>
19. [2412.15248] RoundTripOCR: A Data Generation Technique for Enhancing Post-OCR Error Correction in Low-Resource Devanagari Languages - arXiv, accessed May 16, 2026, <https://arxiv.org/abs/2412.15248>
20. Handwritten Hindi Word Generation to enable Few Instance Learning of Hindi Documents - ECE, IISc, accessed May 16, 2026, <https://ece.iisc.ac.in/~spcom/2020/CameraReadySubmissions/139/CameraReadySubmission/Handwritten-Hindi-Senthil.pdf>
21. Integration of YOLOv8 Small and MobileNet V3 Large for Efficient Bird Detection and Classification on Mobile Devices - MDPI, accessed May 16, 2026, <https://www.mdpi.com/2673-2688/6/3/57>
22. An Integrated Framework with SAM and OCR for Pavement Crack Quantification and Geospatial Mapping - MDPI, accessed May 16, 2026, <https://www.mdpi.com/2412-3811/10/12/348>
23. PaddleOCR-VL: Boosting Multilingual Document Parsing via a 0.9B Ultra-Compact Vision-Language Model - arXiv, accessed May 16, 2026, <https://arxiv.org/html/2510.14528v1>
24. Face Recognition Attendance System using OpenCV - IJFMR, accessed May 16, 2026, <https://www.ijfmr.com/papers/2025/2/40218.pdf>
25. Personal Verification System Using ID Card and Face Photo - ResearchGate, accessed May 16, 2026, https://www.researchgate.net/publication/335143642_Personal_Verification_System_Using_ID_Card_and_Face_Photo
26. Internet of Things with Deep Learning-Based Face Recognition Approach for Authentication in Control Medical Systems - PMC, accessed May 16, 2026, <https://pmc.ncbi.nlm.nih.gov/articles/PMC8858039/>
27. English Handwriting Recognition using Advanced OCR - IJRAR, accessed May 16, 2026, <https://www.ijrar.org/papers/IJRAR23B1440.pdf>
28. Name Entity Recognition for Nepali text using Spacy | by Manoj Kumal | Medium, accessed May 16, 2026, <https://medium.com/@manojkumal/name-entity-recognition-for-nepali-text-using-spacy-eafeea0ac00b>
29. Building an OCR Service with Flask and Python: From Image to Text in Minutes - Medium, accessed May 16, 2026, <https://medium.com/@mimo91/building-an-ocr-service-with-flask-and-python-from-image-to-text-in-minutes-d08cf32af041>
30. Top 7 Open-Source OCR Models in 2025: Supercharge Your Document Digitization with Cutting-Edge Tools! - Neura Market, accessed May 16, 2026, <https://www.neura.market/directories/chatgpt/blog/top-7-open-source-ocr-models-i>

[n-2025-supercharge-your-document-digitization-with-cutting-edge-tools](#)

31. Is there a successful OCR solution for Hindi? - Milvus, accessed May 16, 2026, <https://milvus.io/ai-quick-reference/is-there-a-successful-ocr-solution-for-hindi>
32. Multilingual support · Issue #1699 · mindee/doctr - GitHub, accessed May 16, 2026, <https://github.com/mindee/doctr/issues/1699>
33. Devanagari OCR Techniques and Methods | PDF | Optical Character Recognition - Scribd, accessed May 16, 2026, <https://www.scribd.com/document/424686223/Ocr>
34. Building a Fast Multilingual OCR Model with Synthetic Data - Hugging Face, accessed May 16, 2026, <https://huggingface.co/blog/nvidia/nemotron-ocr-v2>
35. Where do I get a data set for Hindi characters recognition? - Milvus, accessed May 16, 2026, <https://milvus.io/ai-quick-reference/where-do-i-get-a-data-set-for-hindi-characters-recognition>
36. ESC-YOLOv8: An enhanced deep learning framework for semantic understanding of single-line diagram imagery - PMC, accessed May 16, 2026, <https://pmc.ncbi.nlm.nih.gov/articles/PMC12978460/>
37. Uploading Files — Flask Documentation (3.1.x), accessed May 16, 2026, <https://flask.palletsprojects.com/en/stable/patterns/fileuploads/>
38. How to Handle File Uploads in Flask - OneUptime, accessed May 16, 2026, <https://oneuptime.com/blog/post/2026-02-02-flask-file-uploads/view>
39. Designing Production-Scale OCR for India: Multilingual and Domain-Specific Systems, accessed May 16, 2026, <https://arxiv.org/html/2602.16430v1>
40. ID Document OCR — Extract Data from IDs & Passports Automatically | Parsio, accessed May 16, 2026, <https://parsio.io/id-documents/>